AGENTIC AI GOVERNANCE / IMPLEMENTED DEMO

Implementation plan - same-day revision

Version 0.2 | 09 October 2026 | Offline, synthetic and portable

Current implementation replaces the earlier design assumptions for the same-day deadline. This document explains what is implemented and what remains future work.

CONTENTS

1. Confirmed direction
2. Delivered architecture and demonstration scope
3. Same-day handover sequence
4. Work allocation and next build stages
5. Remaining decisions and acceptance

PROJECT REFERENCE / V0.2

1. Confirmed direction

The user confirmed a same-day deadline, a required working demo, four to five participants, Python familiarity, acceptance of the recommended banking scenario and synthetic data, and uncertain enterprise network permissions. The project must be portable into the Infosys environment using approved channels. The target cloud for a later product is not selected.

The immediate approach is an offline prototype with deterministic agents, local static browser assets, Python standard-library services and SQLite. It requires no cloud account, model key, outbound request, third-party runtime package or container installation. This replaces the original four-week build assumption and larger proposed stack for today's delivery.

PROJECT REFERENCE / V0.2

2. Delivered architecture and demonstration scope

The browser controls predefined scenarios through a local application API. The governance engine authenticates demo agent tokens, resolves task authority, applies rules and records decisions. A separate local tool service checks a gateway credential before reading or updating synthetic case data. A masked response returns through the gateway. The management UI shows owners, state, approvals, counts and evidence.

Core demonstration: approved knowledge delegation and masked case read; denied destination; denied out-of-scope case; denied privilege expansion; direct-tool credential rejection; expired and unknown authority; exact-request approval; suspension of a parent followed by child denial. Tests check tool execution counts so a denial is not merely a red label in the UI.

Application-level controls are implemented. Network isolation, real employee identity, production workload identity and agent process isolation are not. Human personas are explicitly simulated. The code must not be represented as a production-ready gateway or compliance certification.

PROJECT REFERENCE / V0.2

3. Same-day handover sequence

First, run the preflight check and automated tests on the presentation machine. Second, start with a fresh data directory. Third, rehearse the six-minute script with Employee, Reviewer and Operator personas. Fourth, verify that the archive contains source, local assets, tests and current documentation without runtime databases or secrets. Fifth, transfer through the approved channel and repeat preflight at the destination.

If approved Python, local sockets or child processes are unavailable, use the handbook and stored test evidence as a presentation fallback and request an approved environment. Do not install unapproved interpreters, change firewall policy or tunnel through restrictions merely to complete the demonstration.

PROJECT REFERENCE / V0.2

4. Work allocation and next build stages

Assign one person to governance/policies, one to integrations/deployment, one to UI and one to tests/documentation/demo. A fifth person can own independent security review and enterprise requirements. Start with the existing test suite so changes preserve behavior.

The next sandbox increment replaces demo personas with enterprise login, adopts a production Python API stack, creates a real agent registration lifecycle, adds a managed database and one approved tool integration. After that, isolate agents and tools, introduce workload identity, enforce egress and use a reviewed policy engine. Add a live model only behind an approved endpoint and data policy.

Cloud dependencies to obtain later: approved hosting, identity integration, secrets/key storage, database, private networking/TLS, logging/retention, build registry and optionally a model service. The application is not deployed to any cloud today and no paid resources are created.

PROJECT REFERENCE / V0.2

5. Remaining decisions and acceptance

Still to confirm: the exact judging rubric; destination Python/browser availability; permission for local loopback and subprocesses; approved transfer method; target cloud and landing zone; real identity/tool integrations; data residency and retention; workload scale, performance targets and operational ownership.

Today's acceptance is bounded: launch locally, execute the demo scenarios, show denied requests cause no tool execution, approve a write once, trace decisions and explain implementation limits. The recorded automated suite passed 25 tests in the development environment. Browser walkthrough and package smoke-test evidence are recorded in TEST_REPORT.md. This is not proof of successful execution inside Infosys.

Official runtime references: https://docs.python.org/3/library/http.server.html and https://docs.python.org/3/library/sqlite3.html. The original technical reference register remains in docs/archive/02_Implementation_Plan.pdf.